

Documentação técnica da Samsung Xclipse 940

Projeto: XO940 **Hardware de referência:** Samsung Xclipse 940 no Samsung SM-S721B

Plataforma: s5e9945 / erd9945 **Revisão:** 2026-09-17

1. Resumo

Esta documentação consolida observações de runtime, Device Tree, kernel SGPU/DRM, memória, IOMMU, execução, Android e fontes públicas Samsung. O objetivo é permitir que outra pessoa reproduza as coletas sem depender de conclusões informais ou de logs não classificados.

A base confirma uma infraestrutura SGPU funcional em processos Android suportados. Ela também confirma nós DRM, operações de BO/VA/VM, atividade de scheduler e caminhos de sincronização. A execução independente com resultado validado permanece uma questão separada e deve ser testada com workload isolado.

2. Procedimento de coleta

As coletas foram feitas em aparelho Samsung real, com identificação de build, kernel, SoC, GPU, revisão e firmware. O procedimento usa Device Tree em runtime, sysfs, `/dev/dri`, inventário de bibliotecas, processos Android, tracepoints do kernel, análise de símbolos e probes reversíveis.

Root amplia a observabilidade, mas não elimina namespaces, permissões, SELinux ou dependências do Android. Por isso, cada experimento registra processo, namespace, nó DRM, comando, saída, timestamp, recuperação e hash.

3. Política de evidências

Os resultados são classificados como **Confirmado**, **Confirmado por fonte**, **Parcial**, **Indício**, **Hipótese** ou **Negativo específico**. Um inicializador de teste não é tratado como execução. Uma enumeração de API não é tratada como processamento. Um submit aceito não é tratado como execução concluída sem sincronização e readback.

Para uma afirmação de execução GPU, é necessário demonstrar o dispositivo-alvo, o recurso visível para a GPU, a submissão, a conclusão e a alteração esperada em um buffer ou superfície.

4. Identidade e Device Tree

O nó principal observado é `/sys/firmware/devicetree/base/sgpu@22200000`, com `compatible = samsung-sgpu,samsung-sgpu`. A ramificação inclui `gpu_pm`, `gpu_doorbell`, `gpu_debug`, `gpu_smtarg` e `gpu_sysreg`. As regiões observadas incluem `gpu`, `doorbell`, `debug`, `pwrctl`, `sysreg` e `htu`.

As interrupções são `SGPU` e `GPU-AFM`. O nó referencia o domínio `pd_g3dcore`. Também foram observados recursos relacionados a DVFS, IFPO, thermal G3D, `sgpu_rmem`, `gpu_buffer_dma_heap`, `BTS G3D` e grupos `SysMMU/IOMMU`.

A Device Tree descreve recursos e relações da plataforma. Ela não define sozinha o formato completo dos comandos, a ABI de userspace, a ISA ou a semântica de cada registrador.

5. DRM e topologia

O caminho observado utiliza render nodes DRM. A coleta direta de versão identificou `card0` e `renderD128` com a camada reportada como `amdgpu`, enquanto `card1` e `renderD129` reportaram `exynos-drmdpu`. Esse valor é uma observação do ioctl DRM e precisa ser correlacionado com sysfs, major/minor, driver associado e platform device.

A topologia correta de estudo é:

```
nó DRM → major/minor → sysfs → driver → platform device → Device Tree SGPU
```

O nome retornado pelo ioctl não é suficiente para inferir identidade física, ISA ou compatibilidade binária.

6. Memória, VM e IOMMU

A atividade registrada inclui criação e movimento de BO, mapeamento de CPU, VA map/unmap, atualização de PTE, atualização de page tables e flush de VM. Um probe completou criação de BO, mapeamento, toque de CPU, VA map/unmap e limpeza.

O contrato operacional a ser reproduzido é:

BO / DMA-BUF

- mapeamento SGPU
- atualização de VM / PTE
- flush
- submissão
- execução no ring
- fence / retire
- readback

A existência de endereço ou flag de page table não revela sozinha política de cache, residency, coerência ou mecanismo de recuperação de fault.

7. Submissão e sincronização

Os traces do caminho Android suportado registraram `amdgpu_cs_ioctl`, execução de scheduler e `amdgpu_ib_schedule` no ring `gfx_0.0.0`, com `num_ibs=3`. Outros eventos relacionam mapeamento PIO, operação de BO durante CS e flush de VM ao agendamento do job.

Esse resultado confirma atividade de submissão no caminho observado. Ele não confirma, por si só, interpretação independente do IB, fence de um cliente isolado, escrita em buffer controlado ou readback. A próxima captura deve manter PID, contexto, seqno, timestamp e assinatura do buffer.

8. Android, loader e namespaces

O processo Android suportado mapeia o loader do sistema, bibliotecas vendor e mantém descritores para o render node. A cadeia observada é:

SurfaceFlinger / RenderThread

- loader Android
- bibliotecas gráficas vendor
- libdrm_sgpu
- renderD128
- kernel SGPU

Um ambiente de terminal opera em namespace diferente e pode expor apenas uma implementação de software. Root e mudança temporária de SELinux não garantem ingresso no namespace vendor nem acesso equivalente às bibliotecas do processo suportado.

9. Vulkan, OpenCL e bibliotecas

O inventário identifica `vulkan.samsung.so`, `libdrm_sgpu.so`, `lib0openCL.so` e `libSGPU0openCL.so` em caminhos vendor. Exports de criação de programa, enqueue e leitura de buffer são superfícies úteis para estudo, mas podem depender de inicialização privada, permissões, bibliotecas auxiliares, manifests e formatos internos.

A presença de uma biblioteca ou de um símbolo é classificada como indício de superfície instalada. A execução só é confirmada quando há dispatch, sincronização e resultado independente.

10. Fontes Samsung e variantes

O pacote recebido contém referências a conjuntos SM-S721B, SM-S721U, SM-S7210, SM-S721Q e SM-S721J, incluindo variantes de Device Tree. A comparação deve ser feita por propriedades técnicas: nós SGPU, compatibilidade, power-domain, clocks, IRQ, IOMMU, DMA heap, firmware e revisões.

Os pacotes de fonte são referência de proveniência. Código, firmware, bibliotecas vendor e arquivos gerados não entram na release sem inventário de licença e autorização de redistribuição.

11. XLIA e outros conjuntos

O pacote também contém coletas do projeto XLIA e experimentos de subsistemas Exynos/NPU. Esses materiais incluem energia, térmica, bateria, RAM, processos, NNAPI e sessões Android. Eles não são misturados ao corpus da GPU Xclipse porque possuem objetivos, APIs e riscos de privacidade diferentes.

Logs brutos de telefonia, bateria, identificadores, caminhos de instalação e estado de usuário permanecem fora do Git. Somente resultados técnicos derivados e sanitizados podem ser reutilizados em uma documentação específica.

12. NPU Exynos e NNAPI

O pacote mais recente contém um conjunto separado de experimentos de aceleração neural. O ramo utiliza NNAPI e o dispositivo `enn`; ele não utiliza o render node DRM da GPU Xclipse e

não deve ser confundido com execução SGPU. A nova coleta também testou o caminho com UID comum, sem root.

Os resultados sanitizados confirmam execução de operações simples e de alguns grafos quantizados com saída correlacionada por checksum. Uma cadeia INT8 de quatro camadas `FULLY_CONNECTED` registrou 1,1148 ms por execução no ENN e 6,2407 ms na referência de CPU, com checksum igual e razão aproximada de 5,60×. Uma soma elementar produziu `[11, 22, 33, 44]` conforme esperado.

O desempenho não foi uniformemente superior. Em `FULLY_CONNECTED` de ponto flutuante, o ENN registrou 1,2092 ms, enquanto a referência CPU registrou 0,1332 ms. Em softmax de 1024 elementos, ambas as saídas somaram 1,0000, mas o ENN registrou 1,0264 ms contra 0,0104 ms da CPU. Esses números são resultados dos workloads específicos e não representam uma característica geral da NPU.

Os testes também encontraram limites de compilação em `BATCH_MATMUL`, atenção fundida, grafos com múltiplas saídas e certas ordens de declaração de operandos. Em uma comparação estrutural, entradas declaradas antes de operandos compartilhados compilaram, enquanto a ordem inversa falhou em casos equivalentes. Isso é uma dependência observada do runtime/compiler NNAPI e precisa de reprodução independente antes de ser generalizada.

12.1. Execução NNAPI/ENN sem root

A coleta de ¹⁸/₀₉ confirmou que um processo sem root consegue obter o serviço público `android.hardware.neuralnetworks.IDevice/enn`. O teste encontrou os dispositivos `enn` e `nnapi-reference`, finalizou um modelo, compilou-o no ENN e executou `SOFTMAX`. A saída `[0.032059, 0.087144, 0.236883, 0.643914]` coincidiu com o valor esperado, com diferença máxima `0.000000`.

Essa prova estabelece uma rota rootless para a cadeia `aplicativo → NNAPI → HAL ENN → NPU`, limitada aos grafos reproduzidos. Ela não demonstra que a interface proprietária `vendor.samsung_slsi.hardware.enn_aidl.IEnnInterfaceAidl/default` possa ser usada por um aplicativo. Essa interface retornou nulo nos probes realizados. O acesso direto a `/dev/vertex10` também não é rootless: o UID comum recebeu `Permission denied`, e os probes privilegiados testados não produziram um ioctl funcional.

O novo teste muda a classificação de `SOFTMAX`: ele é funcional no grafo pequeno reproduzido, mas continua lento no benchmark anterior de 1024 elementos. `BATCH_MATMUL` permanece não suportado pelo dispositivo `enn` no formato testado. A documentação detalhada está em [NPU/ENN rootless](#).

O pipeline de decodificação apresentou uma configuração com grafos Q/K/V separados e grafos híbridos aceitos, além de uma tentativa multi-saída rejeitada. O teste sustentado de 90 segundos registrou tempos por janela e frequências de CPU durante as fases CPU e NPU.

Como a telemetria térmica completa não estava disponível, essa coleta não prova throttling; ela fornece um baseline para uma captura futura.

A cadeia própria da NPU é:

```
Aplicação ou teste NNAPI
↓
NNAPI reference / dispositivo ENN
↓
modelFinish + createForDevices + compilationFinish
↓
grafo aceito e executado
↓
checksum / saída numérica / tempo
↓
comparação com CPU e análise térmica
```

12. Ramificação técnica

A estrutura completa está no documento separado de [mapa e ramificação técnica](#). A representação é:

```
SoC Exynos / Android
├─ Xclipse 940 / G3D
│   ├── Device Tree / power / clocks / IRQ / reset
│   ├── SGPU / DRM / UAPI
│   ├── GEM / BO / VA / VM / IOMMU
│   ├── context / CS / IB / BO_LIST
│   ├── scheduler / rings / fences / retire
│   ├── Android loader / namespaces / SELinux
│   └─ reprodução / variantes / investigação de integração
```

13. Estado e próximos experimentos

O próximo ciclo deve correlacionar cada nó DRM ao platform device, fechar o contrato BO/VM/IOMMU, isolar um job com buffer conhecido, correlacionar fence e retire, comparar Device Trees e mapear a fronteira entre loader, namespaces e ICD layers.

Cada experimento deve ter controle negativo, guard regions quando aplicável, limpeza explícita e resultado classificado. Falhas de permissão, `noexec`, namespace ou ausência de tracepoint devem ser registradas como limites da coleta, não como falhas genéricas da GPU.

14. Proveniência e publicação

O pacote recebido em 2026-09-17 possui SHA-256

`bd81f26cee79176805c983e639f3d341c48f01cf52680362c2de394818b4aec1` . A

atualização NPU de 2026-09-18 possui SHA-256

`873a0f8d3ff4d8e9a4168aaf0349f5becfa539cfc9b03d8a7033e3acb08af9` . Os arquivos brutos permanecem fora do repositório. A release pública contém documentação técnica, mapas, imagens, relatório rootless e hashes selecionados; não contém logs crus, dumps, blobs, bibliotecas vendor, firmware, credenciais ou dados pessoais.

Referências